

Machine Learning

Author: Refat Othman

Overview & Agenda

- **What is Machine Learning?**

Definition and the three learning paradigms:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

- **Supervised Learning (Classification & Regression)**

- Core concepts
- Algorithms: k-Nearest Neighbors, Perceptron, Linear Regression
- Real-world examples

- **Model Evaluation & Tuning**

- Loss functions
- Overfitting vs. Underfitting
- Regularization
- Cross-validation

- **Reinforcement Learning**

- Learning via rewards
- Practical examples

- **Unsupervised Learning**

- Clustering and pattern discovery
 - Examples
-

What is Machine Learning?

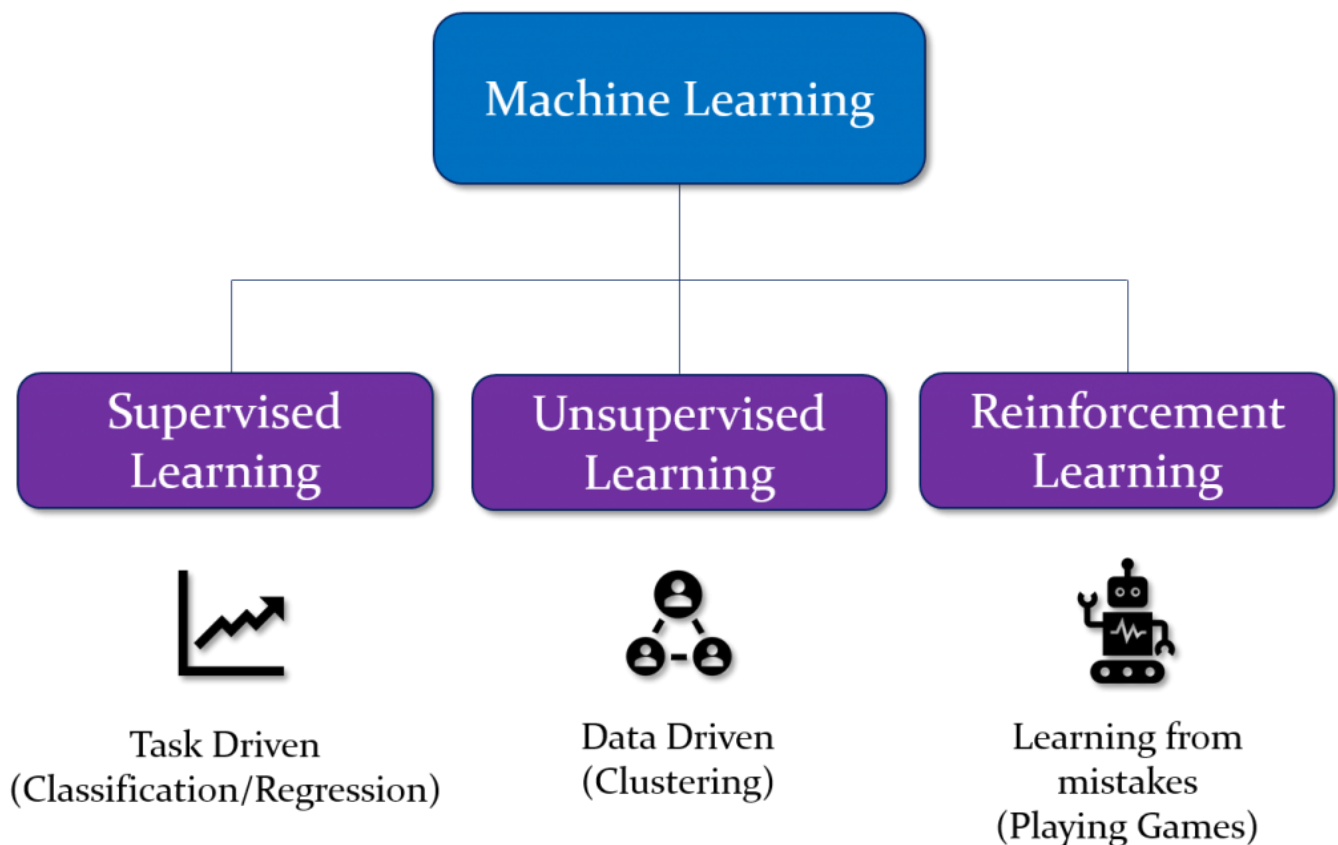
- **Definition:**

Programming computers to learn from data, instead of being explicitly programmed.

- **Key idea:**

The system improves its performance on a task over time as it gains experience (data).

Types of Machine Learning



- **Types of ML:**

- Supervised Learning (learning with a teacher)
- Unsupervised Learning (learning without a teacher)
- Reinforcement Learning (learning by trial and error with feedback)

- **Examples in daily life:**

- Email spam filters
- Voice assistants
- Product recommendations
- Weather apps predicting temperature

Notes

Machine Learning is a subfield of artificial intelligence where we teach computers to learn from data. Instead of writing a rigid program with fixed rules for every scenario, we give the computer examples and let it infer rules or patterns.

A classic definition is that a machine learns if it can improve its performance on a task with experience (more data).

For instance, rather than manually coding how to recognize spam emails, we can train a model on many labeled emails (spam vs. not spam), and it learns to classify new emails.

The Three Main Types of Machine Learning

- **Supervised Learning**

Think of this as learning with a teacher.

We provide the algorithm with:

- Input data
- Correct output labels (answers)

Over time, the model learns to map inputs to outputs.

Example: Training a model on thousands of emails labeled *spam* or *not spam*.

- **Unsupervised Learning**

Learning without a teacher.

The data has **no labels**, and the model must find patterns or structure on its own.

Example: Grouping users based on browsing behavior without knowing the group names in advance.

- **Reinforcement Learning**

Learning by **trial and error** with feedback in the form of rewards or penalties.

Analogy: Learning to ride a bike—trying actions, falling or staying upright, and gradually improving.

Real-World Examples and Their Machine Learning Types

- **Email spam filters → Supervised Learning (Classification)**

- Emails are labeled as *spam* or *not spam*
- The model learns from these labeled examples
- Output: a category (spam / not spam)

- **Voice assistants (Siri, Alexa) → Supervised Learning + Reinforcement Learning**

- Speech recognition is trained on labeled audio–text pairs (Supervised Learning)
- The system improves responses over time using user feedback (Reinforcement Learning)

- **Recommendation systems (Netflix, YouTube, Amazon) → Unsupervised Learning + Supervised Learning**

- Unsupervised Learning: finds patterns and groups users with similar behavior
- Supervised Learning: predicts what content a user is likely to click or watch
- Often combines multiple ML techniques

- **Weather prediction models → Supervised Learning (Regression)**

- Trained on historical weather data
- Input: past temperatures, humidity, pressure
- Output: numeric values (e.g., temperature, rainfall)

Supervised Learning

Supervised Learning Basics

- **Definition:**

Learning from labeled examples (input–output pairs) to predict outputs on new inputs.

- **Goal:**
Find a function (model) that maps inputs to correct outputs by generalizing from training data.
- **Two main tasks:**
 - **Classification:** Predicting categories
 - **Regression:** Predicting numeric values

Notes

In supervised learning, the training data consists of pairs of inputs and desired outputs.

The term *supervised* means that each example comes with a correct answer (label) that guides the learning process.

The algorithm's task is to learn a general rule (model) that maps inputs to outputs so it can make accurate predictions on **new, unseen data**.

Two Types of Supervised Learning Tasks

- **Classification**
The output is a discrete category or class.
Example:
 - Spam vs. not spam
 - Fraud vs. normal transaction
- **Regression**
The output is a continuous numeric value.
Example:
 - House price prediction
 - Temperature forecasting

Think of supervised learning as a student studying solved exercises.

The goal is not to memorize answers, but to **generalize** and solve new problems correctly.

A good supervised model performs well not just on training data, but on **new data it has never seen before**—that is the true test of learning.

Classification: Predicting Categories

Definition

Classification is the task of learning a function that maps an input to a **discrete category (class label)**.

Examples

- **Spam detection:** email → *spam / ham*
- **Image recognition:** image → *dog / cat / other*
- **Weather type:** conditions → *rainy / sunny*

Decision Boundaries

In classification, the model learns a **decision boundary** in the feature space that separates different classes.

- The boundary can be:
 - A **line** (linear models)
 - A **plane** (higher dimensions)
 - A **curve** (non-linear models)

All points on one side of the boundary are assigned to one class, and points on the other side are assigned to another class.

Algorithm Example 1: Nearest Neighbor Classification

Concept

Nearest Neighbor classification works as follows:

1. Store all training examples
2. For a new input, find the **closest** training example
3. Assign the same class label

This is called **1-Nearest Neighbor (1-NN)**.

An extension is **k-Nearest Neighbors (k-NN)**:

- Look at the k closest neighbors
 - Predict the class by **majority vote**
-

Simple Intuition Example

If the closest email you have ever seen was spam, then the new email is also likely spam.

Python Example: Nearest Neighbor (from scratch)

```
import numpy as np

# Training data (two features per example)
X_train = np.array([
    [1, 1], # spam Number of suspicious words, Number of links in email
    [1, 2], # spam
    [4, 4], # ham
    [5, 4]  # ham
])

y_train = ["spam", "spam", "ham", "ham"]

# New point to classify
```

```
x_new = np.array([2, 2])

# Compute Euclidean distances
distances = np.linalg.norm(X_train - x_new, axis=1)

# Find nearest neighbor
nearest_index = np.argmin(distances)

print("Predicted class:", y_train[nearest_index])
```

The model does not learn weights

It memorizes data

Distance = similarity

scikit-learn and k-Nearest Neighbors (k-NN)

1. What is scikit-learn?

scikit-learn (sklearn) is a Python library for classical Machine Learning.

It provides ready-to-use tools for:

- Classification
- Regression
- Clustering
- Dimensionality reduction
- Model evaluation
- Data preprocessing

scikit-learn is built on top of:

- NumPy
- SciPy
- Matplotlib

It is widely used in:

- Education
- Research
- Industry prototypes

2. scikit-learn Design

All machine learning models in scikit-learn follow the same workflow:

1. Create the model

2. Train the model using `fit()`
3. Make predictions using `predict()`
4. Evaluate the model

This unified API makes learning multiple ML algorithms easy.

3. What is k-Nearest Neighbors (k-NN)?

k-Nearest Neighbors (k-NN) is a supervised learning algorithm used for classification and regression.

Key idea:

A data point is classified based on the majority label of its **k closest neighbors**.

4. Properties of k-NN

- Lazy learning algorithm (no training phase)
 - Non-parametric (no learned parameters)
 - Uses distance to make predictions
 - Decision boundaries can be non-linear
 - Sensitive to feature scaling
-

5. Training Data and Features

Each training example is represented as a **feature vector**.

Example training data:

```
X_train = [[1, 1], [1, 2], [4, 4], [5, 4]]
y_train = ["spam", "spam", "ham", "ham"]

#Number of samples: 4
#Number of features: 2

from sklearn.neighbors import KNeighborsClassifier

X_train = [[1, 1], [1, 2], [4, 4], [5, 4]]
y_train = ["spam", "spam", "ham", "ham"]

model = KNeighborsClassifier(n_neighbors=3)
# n_neighbors = 3 means the model looks at the 3 nearest points
# The final class is chosen by majority voting
model.fit(X_train, y_train)
# k-NN does not learn parameters.
# The model simply stores the training data.
prediction = model.predict([[2, 2]])
print("Predicted class:", prediction[0])
```

```
#Compute distance to all training points
# Select the 3 closest points
# Perform majority voting
# Return predicted class
```

Advantages and Disadvantages Advantages

Simple and intuitive

- No training phase
- Works well for small datasets

Disadvantages

- Slow for large datasets
- Sensitive to noise
- Requires feature scaling

When to Use k-NN

Use k-NN when:

- Dataset is small
- You need a baseline model
- Distance-based similarity is meaningful

Avoid k-NN when:

- Dataset is very large
- Real-time prediction is required

Algorithm Example 2: Perceptron (Linear Classifier)

1. Concept

The **Perceptron** is one of the earliest machine learning algorithms inspired by neural networks.

It works by:

- Learning a **weight** for each feature
- Computing a **weighted sum**
- Applying a **threshold (step function)** to make a decision

The Perceptron is a **linear classifier**, meaning it separates data using a straight line (or plane).

2. Intuition

Think of the Perceptron as a **decision maker**:

Each feature gives a vote (weight).
All votes are summed.
If the result is positive → Class 1
Otherwise → Class 0

3. Mathematical Idea

For an input feature vector:

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

The Perceptron computes:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Decision rule:

output =

- 1 if $(\mathbf{w} \cdot \mathbf{x} + b) > 0$
- 0 otherwise

Where:

- $\mathbf{w}$ = weight vector
 - b = bias
 - $\mathbf{x}$ = input features
-

4. Linear Decision Boundary

The equation:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Defines a **linear decision boundary**:

- Line in 2D
 - Plane in 3D
 - Hyperplane in higher dimensions
-

5. Training Data and Labels

```
X_train = [  
    [1, 1],  
    [1, 2],  
    [4, 4],  
    [5, 4]  
]  
#Each row is a feature vector  
# Each column is a feature  
  
# Labels: 0 = spam, 1 = ham  
y_train = [0, 0, 1, 1]  
# Labels are numeric, not text  
  
from sklearn.linear_model import Perceptron  
  
model = Perceptron()  
# Initialize weights randomly (small values)  
# Initialize bias  
# Set learning rate and number of epochs (default)  
# At this moment:  
# The model knows nothing about the data  
  
model.fit(X_train, y_train) #This is where learning happens.  
  
prediction = model.predict([[2, 2]])  
print("Predicted class:", "ham" if prediction[0] == 1 else "spam")
```

Step-by-Step Perceptron Training Example (Inside `fit()`)

This example shows **exactly what happens to the weights and bias** during Perceptron training.

We use simple numbers so students can follow every step.

Training Data

We have **2 features** and **binary labels**:

Sample	x_1	x_2	y (class)
A	1	1	0 (spam)
B	1	2	0 (spam)
C	4	4	1 (ham)

Sample	x_1	x_2	y (class)
D	5	4	1 (ham)

Step 0: Initialization (Before Training)

Assume the Perceptron starts with:

- Weights:
 $w = (0, 0)$
- Bias:
 $b = 0$
- Learning rate:
 $\eta = 1$

At this point, the model **knows nothing** about the data.

Step 1: Process Sample A $\rightarrow x = [1, 1], y = 0$

Compute weighted sum

$$z = (0 \times 1) + (0 \times 1) + 0 = 0$$

Prediction

$$z \leq 0 \rightarrow \text{predicted class} = 0$$

Compare with true label

- Predicted: 0
- True: 0

☒ Correct $\rightarrow$ **No update**

Weights and bias remain:

$$w = (0, 0), b = 0$$

Step 2: Process Sample B $\rightarrow x = [1, 2], y = 0$

Compute weighted sum

$$z = (0 \times 1) + (0 \times 2) + 0 = 0$$

Prediction

$z \leq 0 \rightarrow \text{predicted class} = 0$

Compare with true label

- Predicted: 0
- True: 0

☒ Correct $\rightarrow$ **No update**

$w = (0, 0), b = 0$

Step 3: Process Sample C $\rightarrow x = [4, 4], y = 1$

Compute weighted sum

$$z = (0 \times 4) + (0 \times 4) + 0 = 0$$

Prediction

$z \leq 0 \rightarrow \text{predicted class} = 0$

Compare with true label

- Predicted: 0
- True: 1

☒ Wrong $\rightarrow$ **Update required**

Step 3 (Update Rule)

For learning, we map labels as:

- Class 0 $\rightarrow -1$
- Class 1 $\rightarrow +1$

So here:

$$y = +1$$

Perceptron update equations

$$w \leftarrow w + \eta \cdot y \cdot x$$

$$b \leftarrow b + \eta \cdot y$$

Apply update

$$w = (0, 0) + 1 \times 1 \times (4, 4) = (4, 4)$$

$$b = 0 + 1 = 1$$

☒ Weights changed because the model made a mistake.

Step 4: Process Sample D $\rightarrow x = [5, 4], y = 1$

Compute weighted sum

$$z = (4 \times 5) + (4 \times 4) + 1$$

$$z = 20 + 16 + 1 = 37$$

Prediction

$$z > 0 \rightarrow \text{predicted class} = 1$$

Compare with true label

- Predicted: 1
- True: 1

☒ Correct $\rightarrow$ **No update**

End of Epoch 1 (One Full Pass)

Final parameters after one epoch:

- Weights: $w = (4, 4)$
- Bias: $b = 1$

The decision boundary has moved to better separate the classes.

Prediction After Training

Predict the class of:

$$x = [2, 2]$$

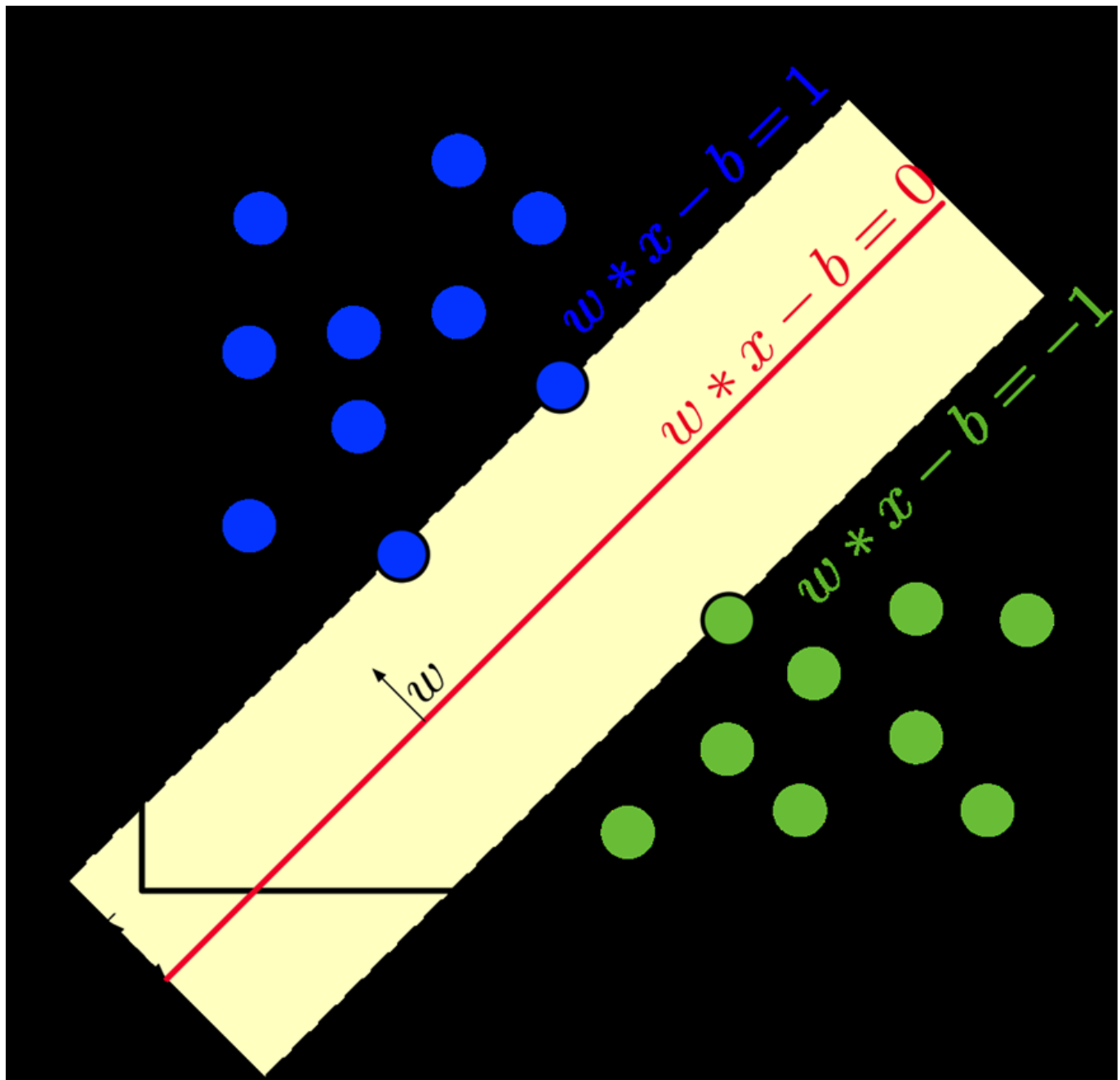
Compute weighted sum

$$z = (4 \times 2) + (4 \times 2) + 1 = 17$$

Prediction

$$z > 0 \rightarrow \text{predicted class} = 1$$

☒ Final prediction: **ham**



Visualizing Decision Boundaries

What is a Decision Boundary?

A **decision boundary** is a surface that separates different classes in the feature space.

- On one side of the boundary → the model predicts **Class A**
- On the other side → the model predicts **Class B**

The **shape of the boundary** depends on:

- The **model** used
- The **complexity** of the model
- The **distribution of the data**

Decision Boundaries with Two Features

When we have **two features**, we can plot the data on a **2D plane**:

- x-axis → Feature 1
- y-axis → Feature 2

In this case:

- The decision boundary is a **line or curve**
- This makes classification **easy to visualize**

In higher dimensions:

- 3 features → boundary is a **plane**
 - More features → boundary is a **hyperplane** (cannot be visualized directly)
-

Linear Decision Boundaries

Linear classifiers learn a **straight-line boundary**.

Examples:

- Perceptron
- Logistic Regression
- Support Vector Machine (linear kernel)

Characteristics:

- Boundary is a straight line in 2D
- Model separates classes using a linear rule:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

This equation defines the decision boundary.

Maximum-Margin Idea (SVM intuition)

Some linear classifiers (like SVM) do more than just separate classes.

They try to:

- Maximize the **margin**
- Margin = distance between the boundary and the nearest data points

Key concepts:

- **Support vectors** → the closest points to the boundary
 - These points “support” or define the boundary
 - A wider margin usually means **better generalization**
-

Visual Interpretation (Linear Case)

- **Red line** → decision boundary
- **Blue points** → Class A
- **Green points** → Class B
- **Gray shaded region** → margin
- **Points touching the margin** → support vectors

If the data is **linearly separable**, a perceptron or SVM can find such a line.

Nonlinear Decision Boundaries

Some models can learn **curved or irregular boundaries**.

Examples:

- k-Nearest Neighbors ($k > 1$)
- Decision Trees
- Neural Networks

Characteristics:

- Boundary bends and adapts to data
- Can separate complex patterns
- Higher flexibility, but higher risk of **overfitting**

k-NN and Irregular Boundaries

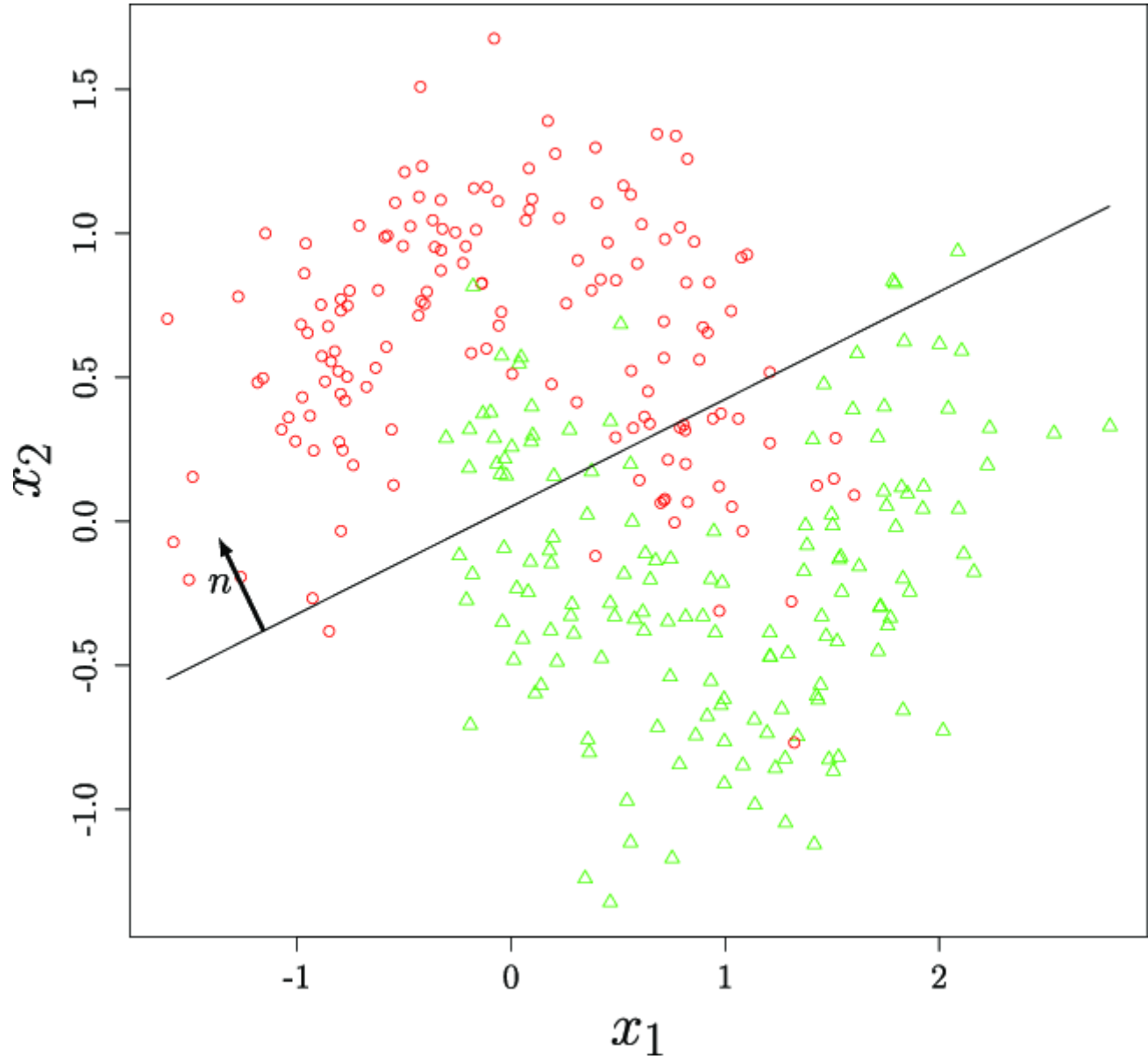
With **k-NN**, the boundary is determined locally:

- Each prediction depends on nearby points
- The boundary can look **jagged or wiggly**
- Increasing k :
 - Smooths the boundary
 - Makes the model more stable
 - But may blur fine distinctions

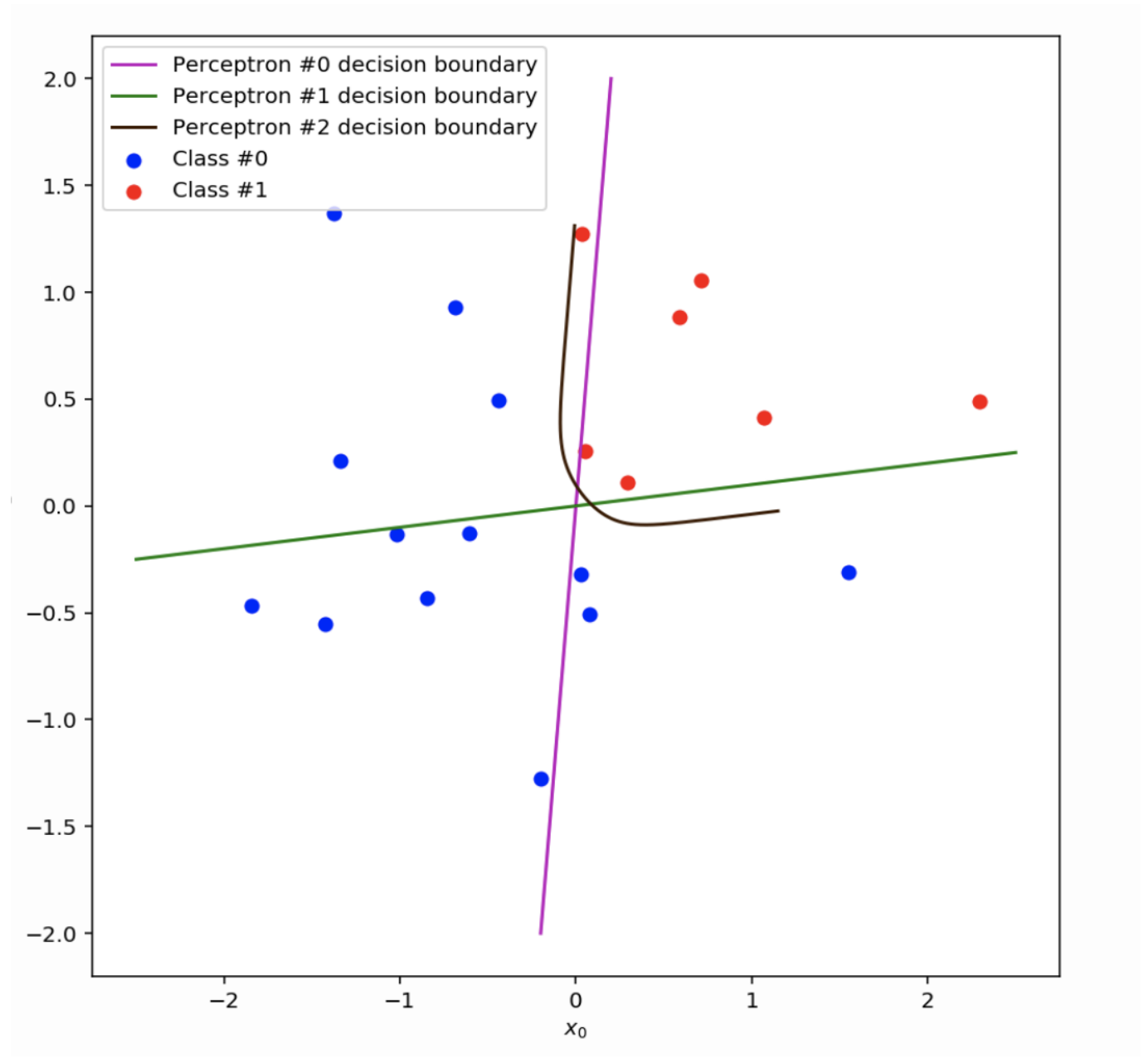
Linear vs. Nonlinear Models (Teaching Comparison)

Model Type	Boundary Shape	Strength	Risk
Linear (Perceptron, SVM)	Straight line	Simple, fast, interpretable	Cannot model complex patterns
k-NN	Irregular	Flexible, intuitive	Sensitive to noise
Decision Trees	Axis-aligned shapes	Easy to explain	Overfitting
Neural Networks	Very complex	Powerful	Hard to interpret

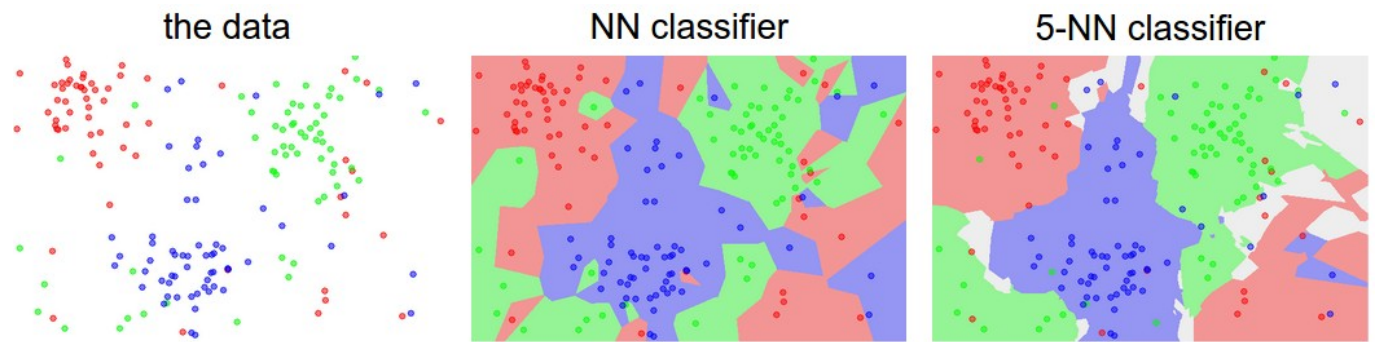
Perceptron



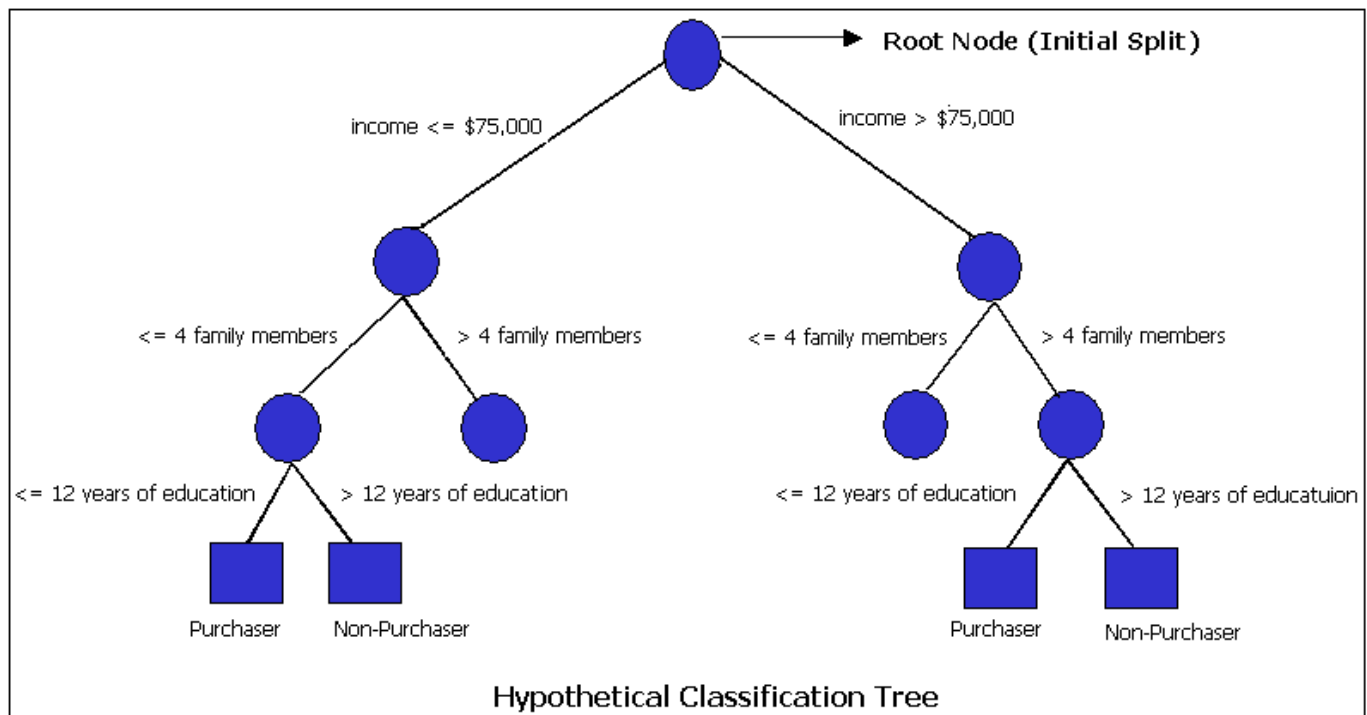
SVM



knn

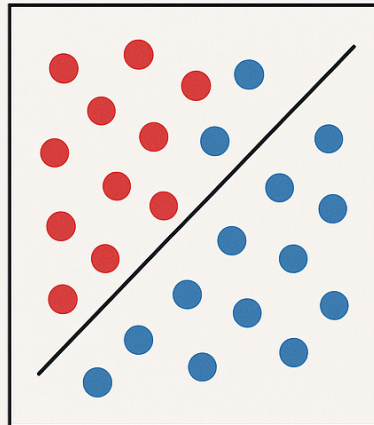


Decision Trees

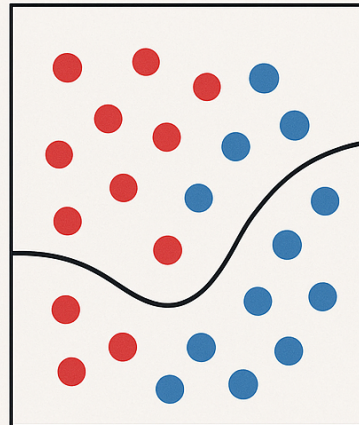


Neural Networks

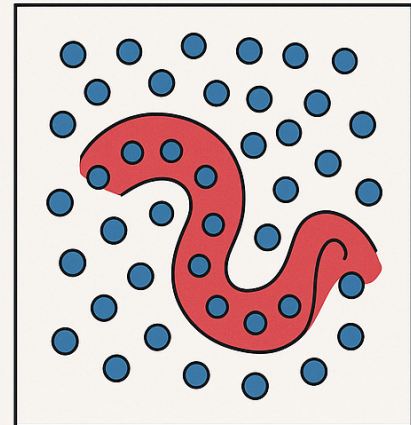
What Do Decision Boundaries Tell Us About a Model?



Linear Classifier



SVM



Neural Network

Notes

"A decision boundary is like a fence that separates different groups.

Linear models build straight fences.

Complex models build curved fences that follow the data."

- Decision boundaries separate classes
- Shape depends on the model
- Linear models → simple, straight boundaries
- Complex models → flexible, curved boundaries
- Good boundaries generalize well to new data

Takeaways

- The Perceptron updates weights **only when it makes a mistake**
- Each mistake moves the decision boundary
- Large feature values cause larger updates
- Training order (permutation) matters
- The model learns a **linear decision boundary**

So during training, the Perceptron processes each example, checks if the prediction is correct, and updates its weights and bias only when a mistake occurs, gradually learning a linear separator.

Examples in details:

Same Dataset — Same Code Structure — Different Models

This example uses **exactly the same training data and test point** to show how different machine learning models behave.

Training Data (Used by ALL Models)

```
X = [  
    [1, 1],  
    [1, 2],  
    [4, 4],  
    [5, 4]  
]  
  
y = [0, 0, 1, 1]    # 0 = spam, 1 = ham  
  
x_new = [[2, 2]] #Test point  
  
# 1. Perceptron (Linear Model)  
  
from sklearn.linear_model import Perceptron  
  
model = Perceptron()  
model.fit(X, y)
```

```

prediction = model.predict(x_new)
print("Perceptron prediction:", prediction)

# 1. Perceptron (Linear Model)
# What happens in fit() – Step by Step
# 1. Initialize parameters
#   Weights:  $w = (0, 0)$ 
#   Bias:  $b = 0$ 
# 2. Start Epoch 1
#   Take first sample  $x = (1,1)$ ,  $y = 0 \rightarrow$  mapped to  $-1$ 
#   Compute  $z = w \cdot x + b = 0$ 
#   Predict  $-1 \rightarrow$  correct  $\rightarrow$  no update
# 3. Second sample  $x = (1,2)$ ,  $y = -1$ 
#    $z = 0 \rightarrow$  predict  $-1 \rightarrow$  correct  $\rightarrow$  no update
# 4. Third sample  $x = (4,4)$ ,  $y = +1$ 
#    $z = 0 \rightarrow$  predict  $-1 \rightarrow$  WRONG
#   Update:
#      $w = w + y \cdot x = (0,0) + (4,4) = (4,4)$ 
#      $b = b + y = 1$ 
# 5. Fourth sample  $x = (5,4)$ ,  $y = +1$ 
#    $z = 4 \cdot 5 + 4 \cdot 4 + 1 = 37$ 
#   Predict  $+1 \rightarrow$  correct  $\rightarrow$  no update
# 6. End of epoch
#   Final  $w = (4,4)$ ,  $b = 1$ 
#   ↻ Learning happens only when a mistake occurs
# What happens in predict()
# 1. Compute  $z = w \cdot x_{\text{new}} + b$ 
#    $z = 4 \cdot 2 + 4 \cdot 2 + 1 = 17$ 
# 2.  $z > 0 \rightarrow$  predict class 1 (ham)
# ✗ No learning
# ✗ No updates

```

```

# 2. k-Nearest Neighbors (k-NN)

```

```

from sklearn.neighbors import KNeighborsClassifier

```

```

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

```

```

prediction = model.predict(x_new)
print("k-NN prediction:", prediction)

```

```

# What happens in fit() – Step by Step

```

```

# 1. Store all training samples X in memory

```

```

# 2. Store labels y

```

```

# 3. Set  $k = 3$ 

```

```

# ✗ No weights

```

```
# ✗ No bias
# ✗ No optimization
# ☞ No learning at all

# What happens in predict() – Step by Step

# 1. Compute distance from x_new = (2,2) to each training point:

# (1,1) → distance ≈ 1.41

# (1,2) → distance = 1

# (4,4) → distance ≈ 2.83

# (5,4) → distance ≈ 3.61
# 2. Sort distances
# 3. Pick 3 nearest neighbors:
# labels = [0, 0, 1]
# 4. Majority vote:
# class 0 wins
# ☞ Prediction = spam (0)
# ✔ All computation happens here
```

```
-----
# 3. Decision Tree
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X, y)
```

```
prediction = model.predict(x_new)
```

```
print("Decision Tree prediction:", prediction)
```

```
=====
```

```
# We will use the SAME example used before
```

```
# Features: x1, x2
```

```
# Labels: 0 = spam, 1 = ham
```

```
#-----
```

```
# STEP 0: IMPORT THE MODEL
```

```
#-----
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
#-----
```

```
# STEP 1: DEFINE THE TRAINING DATA
```

```
#-----
```

```
# X contains the feature vectors
```

```
# Each row is one sample
```

```
# Each column is one feature
```

```
X = [
```

```
    [1, 1], # spam
```

```
    [1, 2], # spam
```

```
    [4, 4], # ham
```

```
    [5, 4]  # ham
```

```
]
```

```
# y contains the class labels
```

```
y = [0, 0, 1, 1]
```

```

# -----
# STEP 2: CREATE THE DECISION TREE MODEL
# -----
# At this moment:
# - No rules exist
# - No splits exist
# - The tree is empty
model = DecisionTreeClassifier()
#-----
# STEP 3: FIT (TRAIN) THE MODEL
# -----
# This is where ALL learning happens for Decision Trees
model.fit(X, y)
# ----- WHAT HAPPENS INSIDE fit() -----
#
# 1) All samples are placed in the ROOT node:
#    [(1,1)->0, (1,2)->0, (4,4)->1, (5,4)->1]
#
# 2) The tree computes impurity of the root:
#    - Number of class 0 = 2
#    - Number of class 1 = 2
#    -  $p(0) = 2/4 = 0.5$ 
#    -  $p(1) = 2/4 = 0.5$ 
#    -  $Gini = 1 - (0.5^2 + 0.5^2) = 0.5$ 
# 3) The tree generates ALL possible split candidates:
#    Feature x1 values: [1, 1, 4, 5]
#    Sorted: [1, 1, 4, 5]
#    Possible splits:
#      x1 <= 2.5 (midpoint between 1 and 4)
#      x1 <= 4.5 (midpoint between 4 and 5)
#
#    Feature x2 values: [1, 2, 4, 4]
#    Sorted: [1, 2, 4, 4]
#    Possible splits:
#      x2 <= 1.5
#      x2 <= 3.0
#
# 4) For EACH split, the tree:
#    - Divides data into left and right nodes
#    - Computes impurity (Gini) for each node
#    - Computes weighted impurity after the split
#
# So the STEP 1: ROOT NODE IMPURITY (BEFORE ANY SPLIT)
# -----
# Total samples = 4
# Class counts:
#   class 0 = 2
#   class 1 = 2
#
# Probabilities:
#    $p(0) = 2/4 = 0.5$ 
#    $p(1) = 2/4 = 0.5$ 
#
#  $Gini(root) = 1 - (0.5^2 + 0.5^2)$ 

```

```

#           = 1 - (0.25 + 0.25)
#           = 0.5
#
# This is our BASE impurity (used to compute Information Gain)

GINI_ROOT = 0.5

# =====
# STEP 2: TRY ALL SPLITS ON FEATURE x1
# =====

# -----
# SPLIT 1: x1 <= 2.5
# (midpoint between 1 and 4)
# -----

# Left node (x1 <= 2.5):
#   (1,1)->0
#   (1,2)->0
#
# Counts:
#   class 0 = 2
#   class 1 = 0
#
# Probabilities:
#   p(0) = 1
#   p(1) = 0
#
# Gini(left) = 1 - (1^2 + 0^2) = 0

GINI_LEFT_25 = 0

# Right node (x1 > 2.5):
#   (4,4)->1
#   (5,4)->1
#
# Counts:
#   class 1 = 2
#   class 0 = 0
#
# Gini(right) = 0

GINI_RIGHT_25 = 0

# Weighted Gini after split:
#   (2/4)*0 + (2/4)*0 = 0

GINI_AFTER_25 = 0

# Information Gain:
#   Gain = GINI_ROOT - GINI_AFTER_25
#         = 0.5 - 0
#         = 0.5

```



```

GAIN_25 = 0.5

# -----
# SPLIT 2: x1 <= 4.5
# (midpoint between 4 and 5)
# -----

# Left node (x1 <= 4.5):
#   (1,1)->0
#   (1,2)->0
#   (4,4)->1
#
# Counts:
#   class 0 = 2
#   class 1 = 1
#
# Probabilities:
#   p(0) = 2/3
#   p(1) = 1/3
#
# Gini(left) = 1 - ((2/3)^2 + (1/3)^2)
#             = 1 - (4/9 + 1/9)
#             = 1 - 5/9
#             = 4/9 ≈ 0.444

GINI_LEFT_45 = 4/9

# Right node (x1 > 4.5):
#   (5,4)->1
#
# Counts:
#   class 1 = 1
#
# Gini(right) = 0

GINI_RIGHT_45 = 0

# Weighted Gini after split:
#   (3/4)*0.444 + (1/4)*0
# = 0.333

GINI_AFTER_45 = (3/4) * (4/9)

# Information Gain:
#   Gain = 0.5 - 0.333
#         ≈ 0.167

GAIN_45 = GINI_ROOT - GINI_AFTER_45

# =====
# STEP 3: TRY ALL SPLITS ON FEATURE x2

```

```

# =====

# -----
# SPLIT 3: x2 <= 1.5
# -----

# Left node:
#   (1,1)->0
#
# Gini(left) = 0

GINI_LEFT_15 = 0

# Right node:
#   (1,2)->0
#   (4,4)->1
#   (5,4)->1
#
# Counts:
#   class 0 = 1
#   class 1 = 2
#
# Probabilities:
#   p(0) = 1/3
#   p(1) = 2/3
#
# Gini(right) = 1 - ((1/3)^2 + (2/3)^2)
#              = 4/9 ≈ 0.444

GINI_RIGHT_15 = 4/9

# Weighted Gini:
#   (1/4)*0 + (3/4)*0.444
# = 0.333

GINI_AFTER_15 = (3/4) * (4/9)

# Information Gain:
#   Gain = 0.5 - 0.333
#         ≈ 0.167

GAIN_15 = GINI_ROOT - GINI_AFTER_15

# -----
# SPLIT 4: x2 <= 3
# -----

# Left node:
#   (1,1)->0
#   (1,2)->0
#
# Gini(left) = 0

```

```

GINI_LEFT_3 = 0

# Right node:
#   (4,4)->1
#   (5,4)->1
#
# Gini(right) = 0

GINI_RIGHT_3 = 0

# Weighted Gini:
#   (2/4)*0 + (2/4)*0 = 0

GINI_AFTER_3 = 0

# Information Gain:
#   Gain = 0.5 - 0
#         = 0.5

GAIN_3 = 0.5

# =====
# STEP 4: COMPARE ALL GAINS AND CHOOSE THE MAX
# =====

# Gains summary:
#   x1 <= 2.5  → Gain = 0.5   (BEST)
#   x1 <= 4.5  → Gain ≈ 0.167
#   x2 <= 1.5  → Gain ≈ 0.167
#   x2 <= 3.0  → Gain = 0.5   (TIED BEST)

# The tree selects ONE of the maximum-gain splits.
# In practice, sklearn picks the first best split it encounters.

# Selected split:
#   x1 <= 2.5

# =====
# FINAL RESULTING TREE
# =====
#
# IF x1 <= 2.5:
#     predict class 0
# ELSE:
#     predict class 1
#
# Training STOPS because:
# - Both leaves are pure
# - Impurity = 0
# - No further improvement possible
#
# =====

```

```

# -----
# STEP 4: PREDICT A NEW SAMPLE
# -----
x_new = [[2, 2]]

prediction = model.predict(x_new)

print("Prediction:", prediction)

# ----- WHAT HAPPENS INSIDE predict() -----
#
# 1) Start at the ROOT node
#
# 2) Apply the learned rule:
#     Is x1 <= 2.5 ?
#
# 3) For x_new = (2,2):
#     2 <= 2.5 → YES
#
# 4) Move to LEFT child node
#
# 5) Left node is a LEAF:
#     Stored class = 0
#
# 6) Output prediction = 0 (spam)
#
# IMPORTANT:
# - No learning happens here
# - No impurity is computed
# - No tree structure is changed
#
# ----- END OF predict() -----

# =====
# FINAL SUMMARY
# =====
#
# - Decision Trees learn ONLY during fit()
# - Learning = building if/else rules
# - Splits come from data values (midpoints)
# - Impurity (Gini) decides the best split
# - predict() only follows the rules
# - No weights, no gradients, no updates during prediction
#
# =====

```

```

# 4. Neural Network (MLP)
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, random_state=1)
model.fit(X, y)

prediction = model.predict(x_new)
print("Neural Network prediction:", prediction)

# =====
# Neural Network (MLP) – FULL STEP-BY-STEP EXPLANATION IN CODE
# (Same example, explained line by line for teaching)
# =====

# -----
# STEP 0: IMPORT THE MODEL
# -----
# MLPClassifier = Multi-Layer Perceptron
# This is a Neural Network implementation in scikit-learn
# It learns by adjusting WEIGHTS using gradient descent
from sklearn.neural_network import MLPClassifier

# -----
# STEP 1: DEFINE THE TRAINING DATA (SAME EXAMPLE)
# -----
# X = input features
# Each row = one sample
# Each column = one feature
X = [
    [1, 1], # spam
    [1, 2], # spam
    [4, 4], # ham
    [5, 4]  # ham
]

# y = class labels
# 0 = spam, 1 = ham
y = [0, 0, 1, 1]

# -----
# STEP 2: DEFINE A NEW SAMPLE TO CLASSIFY
# -----
# This is the SAME test point used for all models
x_new = [[2, 2]]

# -----
# STEP 3: CREATE THE NEURAL NETWORK MODEL
# -----
model = MLPClassifier(
    hidden_layer_sizes=(5,), # ONE hidden layer with 5 neurons

```

```

    max_iter=1000,          # Maximum number of training epochs
    random_state=1          # Fix randomness for reproducibility
)

# At this moment:
# - NO learning has happened yet
# - All weights are initialized RANDOMLY
# - The network predictions are meaningless

# -----
# STEP 4: FIT (TRAIN) THE MODEL
# -----
model.fit(X, y)

# ----- WHAT HAPPENS INSIDE fit() -----
#
# The following steps are REPEATED many times (epochs):
#
# -----
# (1) INITIALIZATION
# -----
# - All weights between neurons are initialized randomly
# - Biases are initialized (usually small values)
#
# Example (conceptual):
#   w1, w2 -> hidden neuron 1
#   w3, w4 -> hidden neuron 2
#   ...
#   w9, w10 -> output neuron
#
# The network does NOT know anything yet.
#
# -----
# (2) FORWARD PASS (PREDICTION ATTEMPT)
# -----
# For each training sample (x, y):
#
# - Multiply inputs by weights
# - Add bias
# - Apply activation function (ReLU / logistic)
#
# Example:
#   z_hidden = W1 · x + b1
#   a_hidden = activation(z_hidden)
#
#   z_output = W2 · a_hidden + b2
#   y_hat = activation(z_output)
#
# This produces a PREDICTION.
#
# -----
# (3) COMPUTE LOSS (ERROR)
# -----

```

```

# The network compares:
#   predicted value (y_hat)
#   true value (y)
#
# It computes a LOSS value:
#   - Large loss = bad prediction
#   - Small loss = good prediction
#
# Example loss:
#   Binary Cross-Entropy
#
# Loss answers:
#   "How wrong is the network?"
#
# -----
# (4) BACKPROPAGATION (CORE IDEA)
# -----
# The network computes GRADIENTS:
#    $\partial \text{loss} / \partial \text{weight}$ 
#
# This tells:
#   - which weights caused the error
#   - how much they contributed
#
# This step uses CALCULUS (chain rule).
#
# -----
# (5) WEIGHT UPDATE (LEARNING STEP)
# -----
# Each weight is updated:
#
#   weight = weight - learning_rate * gradient
#
# Meaning:
#   - If a weight increases error -> decrease it
#   - If a weight reduces error -> increase it
#
# -----
# (6) EPOCH LOOP
# -----
#   - Steps (2) to (5) are repeated for ALL samples
#   - One full pass = one epoch
#   - This repeats until:
#       * loss stops improving OR
#       * max_iter is reached
#
# -----
# IMPORTANT NOTES:
# -----
#   - Learning is CONTINUOUS
#   - Weights change a LITTLE at each step
#   - There are NO rules or splits
#   - Everything is numeric optimization
#

```

```

# ----- END OF fit() -----

# -----
# STEP 5: PREDICT USING THE TRAINED MODEL
# -----
prediction = model.predict(x_new)

print("Neural Network prediction:", prediction)

# ----- WHAT HAPPENS INSIDE predict() -----
#
# 1) Take the learned weights (from training)
# 2) Perform a FORWARD PASS ONLY
# 3) Compute activations layer by layer
# 4) Produce an output score
# 5) Convert score to class label (0 or 1)
#
# IMPORTANT:
# - NO loss is computed
# - NO gradients are computed
# - NO weights are updated
#
# predict() = USE the learned knowledge
#
# ----- END OF predict() -----

# =====
# FINAL SUMMARY
# =====
#
# - Neural Networks learn ONLY in fit()
# - Learning = adjusting weights using gradients
# - fit() = optimization process
# - predict() = forward computation only
# - Same API as other models, but different internal logic
#
# =====

```